Model Training and Development Using YOLOv8n-Seg

To address the second objective of this study, a YOLOv8n-seg model was trained for rice leaf disease instance segmentation. To fulfill this objective, two separate YOLOv8n-Seg models were developed and trained: one dedicated to detecting and segmenting the rice leaf itself, while the other was developed to detect and segment rice leaf disease lesions and Healthy Rice Leaf across the five target classes. The development, training configuration, and performance evaluation of both models are discussed in the following sections.

To start the rice leaf image analysis, the system must first determine whether a rice leaf is present in the captured image. For this task, a YOLOv8n-Seg model was trained on the rice leaf detection dataset described in the data preparation phase. The model was trained across 6 Dataset versions, with progressive adjustments to epoch, batch size, dataset size, and image size to improve detection accuracy and generalization.

```
from ultralytics import YOLO

model = YOLO("yolov8n-seg.pt")

results = model.train(
    data="/kaggle/working/AgriEye-palay-detect-updated-2/data.yaml",
    epochs=200,
    imgsz=640,
    batch=16,
    device="0,1",
    workers=2,
    project="/kaggle/working/runs",
    name="palay-seg-v2",
    patience=20,
    save=True,
    plots=True,
    val=True,
)
```

Figure 13. **Model Training configuration for Palay Detection**

The YOLOv8n-seg model for palay detection was trained using the Ultralytics library, as seen in Figure 13, with the pre-trained weights from yolov8n-seg.pt as the starting point, allowing the model to leverage transfer learning for improved performance. The training was configured with 200 epochs and an early stopping patience of 20, meaning training would halt if no improvement was observed after 20 consecutive epochs. An input image size of 640×640 pixels and a batch size of 16 were used throughout the training process. The training was executed on Kaggle using two GPUs with 2 data loading workers to accelerate the process. The dataset configuration was defined through a data.yaml file pointing to the AgriEye Palay detection dataset, and the training outputs were saved under the project directory as "palay-seg-v2". Validation was enabled during training (val=True), and both model checkpoints and training plots were saved (save=True, plots=True), allowing for comprehensive monitoring and evaluation of the model's learning progress throughout the training process.

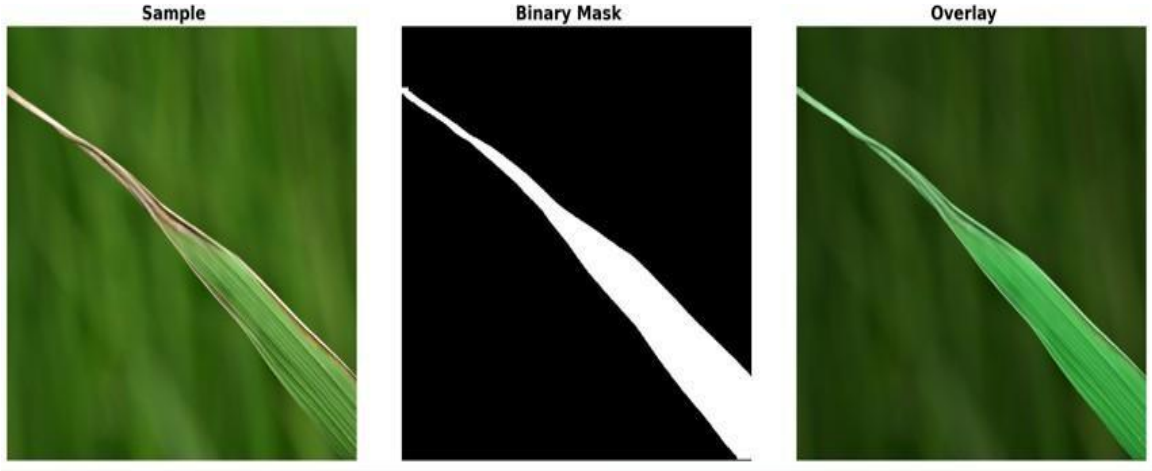


Figure 14. **Sample Leaf Mask**

To perform precise segmentation of the disease-affected areas, a second YOLOv8n-Seg model, the Disease Detection Model, was trained to identify and segment lesions caused by four rice leaf diseases. For each detected lesion, the model produces a pixel-level segmentation mask, which defines the extent of the infected leaf area.

```
from ultralytics import YOLO

FIXED_YAML = 'agrieye_fixed.yaml'

TRAIN_CONFIG = dict(
    data         = FIXED_YAML,
    epochs       = 50,
    imgsz        = 640,
    batch        = 16,        # reduce to 8 if GPU out of memory
    optimizer    = "AdamW",
    lr0          = 0.0001,
    lrf          = 0.01,
    momentum     = 0.937,
    weight_decay = 0.0005,
    patience     = 20,
    cos_lr       = True,
    hsv_h        = 0.015,
    hsv_s        = 0.5,
    hsv_v        = 0.4,
    degrees      = 10.0,
    translate    = 0.1,
    scale        = 0.4,
    flipud       = 0.5,
    fliplr       = 0.5,
    mosaic       = 1.0,
    mixup        = 0.1,
    project      = 'runs/agrieye',
    name         = 'yolov8n_FinalModel#5 - Fine tune',
    exist_ok     = True,
    save_period  = 10,
    plots        = True,
    verbose      = True,
    device       = 0,
    amp          = True,
)
```

Figure 15. **Model Training configuration for Disease Detection**

The training configuration for the disease segmentation model was set up using the Ultralytics YOLO library, as seen in Figure 15, with the dataset defined through agrieye_fixed.yaml. The model was trained for 50 epochs with an image size of 640×640 pixels and a batch size of 16. The AdamW optimizer was used

with an initial learning rate of 0.0001 and a final learning rate of 0.01, alongside a momentum of 0.937 and a weight decay of 0.0005 to regulate overfitting. Cosine learning rate scheduling was also enabled to allow smooth learning rate annealing throughout training. Early stopping was set with a patience of 20 epochs, and model checkpoints were saved every 10 epochs. To improve model generalization, several data augmentation techniques were applied, including HSV color jittering, random rotation up to 10 degrees, translation, scaling, vertical and horizontal flipping, mosaic augmentation at full scale, and mixup at 0.1. The training was conducted on a single GPU on a local device with Automatic Mixed Precision enabled for memory-efficient and faster computation. The training outputs were saved under the project directory runs/agrieye and labeled as "yolov8n_FinalModel#5 - Fine tune", indicating that this configuration was part of a fine-tuning stage of the model development process.

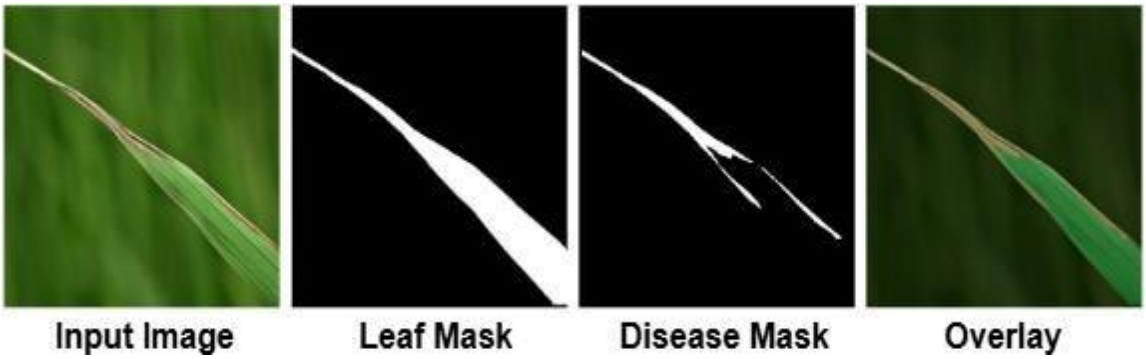


Figure 16. **Sample Leaf and Disease Mask**

YOLOv8n-Seg was selected for both models for the same reason: its lightweight architecture (~2MB) makes it suitable for offline mobile deployment, its single-pass inference design reduces latency on mobile hardware, and its

Table 12. **Performance Evaluation of Disease Detect Model**

Model Ver	Key adjustment	Performance Metrics (P,R,F1,50, 50-95,IoU)	Observations
v1	Epochs:200 Batch: 8 Classes: 6	0.6024 0.5208 0.5586 0.4407 0.2639 0.3389	• Baseline model with moderate performance • High confusion with background, model struggles to separate disease from surroundings • LEAF BLAST and NBS frequently misclassified with each other • HEALTHY class undetected, too few samples or mixed with other classes
v2	Epochs: 300 Batch: 8 Classes: 6	0.7133 0.5954 0.6490 0.5898 0.3447 0.3576	• HEALTHY class detected for the first time, extended training helped • LEAF BLAST and NBS still show heavy confusion with background • Background misclassification remains the primary limiting factor.
v3	Epoch: 100 Batch:8 Classes: 5	0.7132 0.5713 0.6344 0.5903 0.3712 0.4797	• HEALTHY class achieves exceptional IoU (0.833) after Leaf mask removal • Removing Leaf mask reduced class overlap and improved overall clarity • BLS and LEAF BLAST still show notable confusion with background
v4	Epoch: 150 Batch:16 Classes: 4	0.6449 0.4809 0.5509 0.4639 0.2319 0.3601	• Removing the Healthy class reduced model focus, performance drops vs. V3 • Recall drops significantly, model misses many disease instances • Fewer epochs (150) may have prevented the model from fully converging • Results suggest Healthy class provides useful contrast for disease separation
v5	Epoch: 300 Batch: 16 Classes: 4	0.7254 0.5866 0.6486 0.5962 0.3335 0.4739	• Significant improvement over V4, more epochs helped the 4-class model converge • Background misclassification still present but reduced vs. earlier versions • Strong candidate but slightly outperformed by the V8 model.
v6	Epoch: 150 Batch: 16 Classes: 4	0.6888 0.5481 0.6104 0.5660 0.3137 0.4498	• Underperforms V5 across all metrics, fewer epochs limit convergence • Lower recall indicates many disease instances still going undetected • Background confusion remains high for LEAF BLAST and NBS • Shorter training did not benefit this dataset size and class count
v7	Epoch: 50 Batch: 16 Classes: 4	0.6728 0.6059 0.6375 0.6077 0.3287 0.4641	• Slight precision drop vs. V5, more detections include some false positives • Better recall-precision balance than earlier 4-class versions
v8	Epoch:100 Batch: 16 Classes: 5	0.7459 0.6355 0.6862 0.6326 0.4037 0.5461	• Best performance across all versions, highest in all metrics • Reintroducing Healthy class improved disease class separation significantly • Model distinguishes it cleanly • Best localization accuracy of all versions

Table 12 presented the performance evaluation metrics of the disease detection model across eight different model training iterations. Each version was presented with different training parameters, such as the number of classes, epochs, batch size, and dataset composition, to determine the most appropriate and effective configuration for the model. The table presents the evaluation metrics and corresponding performance metrics: Precision, Recall, F1-score, mAP@50, mAP@50-95, and Intersection over Union (IoU). Observations are also presented for each model. These results are to be used to determine the most appropriate model in terms of overall performance for field deployment.

The results presented in Table 12, together with the key observations for each model version, were used to compare the effectiveness of the different training configurations and to identify the model with the best overall performance. With careful consideration for all model results, the researchers were able to determine the configuration that provided the most reliable, accurate model to be used for disease detection.

V8 is selected as the final deployment model. It achieves the highest performance across all evaluated metrics, demonstrates the best localization accuracy (mAP@50-95 = 0.4037), and benefits from a class configuration that reflects real-world field conditions where healthy rice plants are present

YOLOv8n-Seg Model Training (Disease and Leaf Detection)

Environment Setup and Dependency Installation

Description: These commands install the Ultralytics library and verify GPU availability before training begins. ultralytics.checks() confirms that CUDA is accessible and prints hardware information.

```
# -- Install dependencies ----------------------------------------
import subprocess, sys
subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'ultralytics', 'pyyaml', '--quiet'])

# -- Verify GPU --------------------------------------------------
import ultralytics
ultralytics.checks()

# -- Imports used across cells -----------------------------------
from pathlib import Path
import yaml, zipfile
```

Dataset Preparation

Description: This snippet extracts the dataset ZIP archive and rewrites data.yaml with absolute paths so YOLOv8 can locate the training, validation, and test images regardless of the current working directory.

```
import zipfile, yaml
from pathlib import Path

DATASET_DIR = Path('datasets/agrieye')
FIXED_YAML  = 'agrieye_fixed.yaml'
ZIP_PATH    = 'C:\\Users\\Nino\\Downloads\\AgriEye_No leaf with healthy.v2i.yolov8.zip'  # update

# 1. Extract zip
DATASET_DIR.mkdir(parents=True, exist_ok=True)
print(f'Extracting {ZIP_PATH} ...')
with zipfile.ZipFile(ZIP_PATH, 'r') as z:
    z.extractall(DATASET_DIR)
print('Done.')
```

```
# 2. Rewrite data.yaml with absolute paths
YAML_PATH = DATASET_DIR / 'data.yaml'
ABS       = str(DATASET_DIR.resolve())
with open(YAML_PATH) as f:
    cfg = yaml.safe_load(f)

cfg['train'] = f'{ABS}/train/images'
cfg['val']   = f'{ABS}/valid/images'
cfg['test']  = f'{ABS}/test/images'

with open(FIXED_YAML, 'w') as f:
    yaml.dump(cfg, f, default_flow_style=False)

print(f'Fixed yaml saved to: {FIXED_YAML}')
print(yaml.dump(cfg))

# 3. Image check
for split in ['train', 'valid', 'test']:
    imgs = list((DATASET_DIR / split / 'images').glob('*.jpg'))
    lbls = list((DATASET_DIR / split / 'labels').glob('*.txt'))
    print(f'  {split:6s}: {len(imgs)} images, {len(lbls)} labels')
```

Training the YOLOv8n-seg Model

Description: This command trains the YOLOv8 nano segmentation model using the defined configuration dictionary. The yolov8n-seg.pt pretrained COCO weights are used as a starting point for training. A similar code structure is used for transfer learning.

```
from ultralytics import YOLO

FIXED_YAML = 'agrieye_fixed.yaml'

TRAIN_CONFIG = dict(
    data         = FIXED_YAML,
    epochs       = 50,
    imgsz        = 640,
    batch        = 16,        # reduce to 8 if GPU out of memory
    optimizer = "AdamW",
    lr0          = 0.0001,
    lrf          = 0.01,
    momentum     = 0.937,
    weight_decay = 0.0005,
    patience     = 20,
    cos_lr       =True,
    hsv_h        = 0.015,
    hsv_s        = 0.5,
    hsv_v        = 0.4,
    degrees      = 10.0,
    translate    = 0.1,
    scale        = 0.4,
    flipud       = 0.5,
    fliplr       = 0.5,
    mosaic       = 1.0,
```

```
    mixup         = 0.1,
    project       = 'runs/agrieye',
    name          = 'yolov8n_FinalModel#5 - Fine tune',
    exist_ok      = True,
    save_period   = 10,
    plots         = True,
    verbose       = True,
    device        = 0,
    amp           = True,
)

model        = YOLO('YOLOv8n-seg.pt')       # pretrained COCO weights or best.pt for transfer learning
train_result = model.train(**TRAIN_CONFIG)

BEST_WEIGHTS = str(train_result.save_dir / 'weights' / 'best.pt')
RESULTS_CSV  = str(train_result.save_dir / 'results.csv')
print(f'\nTraining complete!')
print(f'Best weights : {BEST_WEIGHTS}')
```

Model Evaluation on Validation and Test Sets

Description: This snippet loads the best checkpoint and evaluates it on both the validation and test splits. It reports mAP50, mAP50-95, Precision, and Recall. Per-class mAP50 is also printed for the validation set to identify which disease classes perform weakest.

```
from ultralytics import YOLO

FIXED_YAML   = 'agrieye_fixed.yaml'
CLASS_NAMES  = ['BLB', 'BLS', 'HEALTHY', 'LEAF BLAST', 'NBS']
BEST_WEIGHTS = 'C:\\Users\\Nino\\Downloads\\runs\\segment\\runs\\agrieye\\yolov8n_FinalModel#5_Stage2\\weights\\best.pt'

model_eval = YOLO(BEST_WEIGHTS)

for split in ['val', 'test']:
    m = model_eval.val(data=FIXED_YAML, split=split, imgsz=640, verbose=False)
    print(f'\n{split.upper()} SET:')
    print(f'  mAP50      : {m.box.map50:.4f}')
    print(f'  mAP50-95   : {m.box.map:.4f}')
    print(f'  Precision  : {m.box.mp:.4f}')
    print(f'  Recall     : {m.box.mr:.4f}')
    if split == 'val':
        print('  Per-class AP50:')
        for name, ap in zip(CLASS_NAMES, m.box.ap50):
            print(f'    {name:15s}: {ap:.4f}')
```

Exporting the YOLOv8n-seg model as TFLite

Description: This command converts the trained .pt model to TensorFlow Lite format for on-device inference in the Android application.

```
# TFLite conversion
model_eval.export(format='tflite', device = 0)

print('\n Model exported!')
```